

SentinelAPI

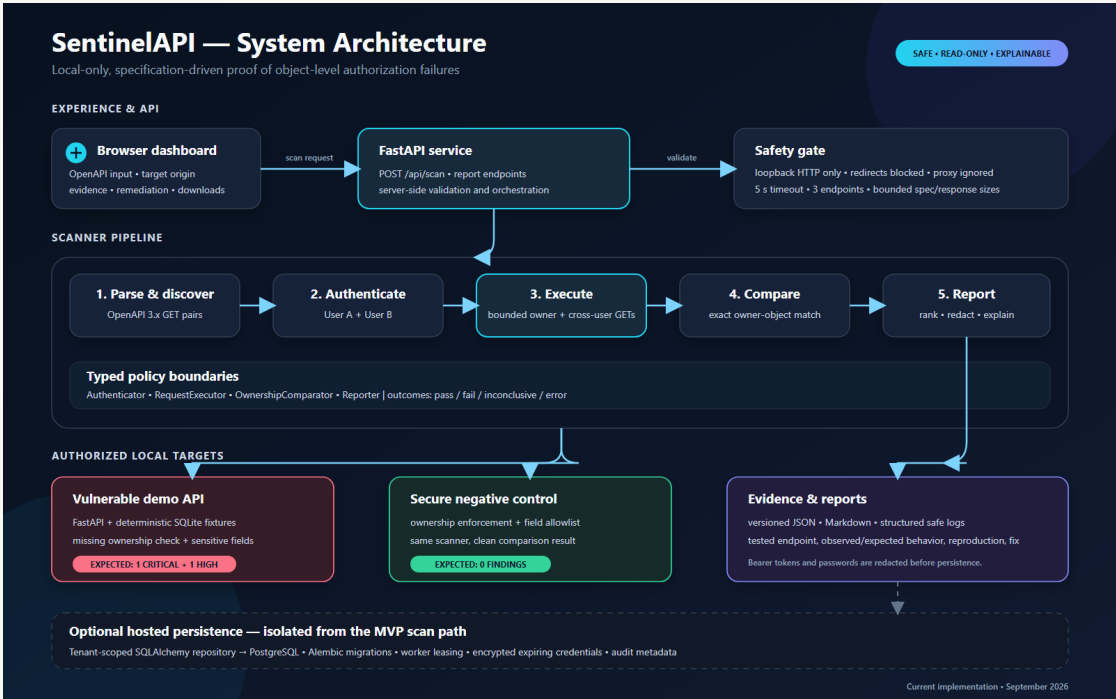
Project, Demo, and Jury Handbook

CORE CLAIM

One authenticated user receives another user's order from live local API responses.

A safe, specification-driven scanner for authorized local sandboxes

This handbook explains the product, end-to-end workflow, technical implementation, safety model, live demonstration, presentation plan, jury questions, failure recovery, limitations, and roadmap.



Current verified test result: 70 tests passing on 25 September 2026. Older repository prose that states 67 tests is stale.

GUIDE MAP

Contents

01	Executive summary and problem
02	Product scope and users
03	How SentinelAPI works
04	Architecture and component responsibilities
05	Detection logic and outcomes
06	Safety and security boundaries
07	Dashboard, reports, and demo evidence
08	Technology stack and repository map
09	Setup, reset, testing, and live demo
10	Seven-minute presentation script
11	Jury questions and answers
12	Where the team can get stuck
13	Limitations, roadmap, contributions, and submission notes

POSITIONING

Present SentinelAPI as a focused authorization proof tool, not as a general autonomous hacking agent or complete OWASP API security platform.

01

Executive summary

SentinelAPI is a local-only API security scanner designed to prove Broken Object Level Authorization, also called BOLA or IDOR. It uses two controlled identities, establishes which object each user owns, requests one user's object with the other user's token, and confirms a vulnerability only when the cross-user response matches the owner's response.

The deliberately vulnerable order API produces one Critical BOLA finding and one High excessive-data-exposure finding. A secure control returns HTTP 403 and produces zero confirmed findings. The scanner also generates severity-ranked, redacted JSON and Markdown reports with expected behavior, observed evidence, reproduction steps, and remediation.

PRODUCT STATEMENT

A safe, specification-driven scanner that proves whether one authenticated API user can access another user's data, then gives developers the evidence needed to reproduce and fix it.

What problem does it solve?

- A valid token proves identity, but it does not prove ownership of every requested object.
- Generic signature scanners can miss this business-logic failure because the request is syntactically valid.
- Developers need endpoint-specific evidence, expected behavior, reproduction, and remediation rather than raw logs.
- Security teams need a safe, repeatable demonstration with low false-positive noise.

Who is it for?

- Backend and platform engineers who own API authorization logic.
- Application security engineers who need reproducible evidence.
- Engineering managers and compliance stakeholders who need a clear severity-ranked report.

PROBLEM MODEL

Authentication, authorization, BOLA, and exposure

Term	Meaning in SentinelAPI
Authentication	Confirms which controlled user is calling the API.
Authorization	Confirms whether that user may access the specific requested object.
BOLA / IDOR	A caller changes an object identifier and receives another user's object because ownership was not enforced.
Owner baseline	The object returned to its legitimate owner and used as comparison evidence.
Cross-user request	User A's token requesting the object ID selected for User B.
Excessive data exposure	An ordinary user response includes internal or secret fields that should be omitted.

The deterministic example

- User A is user-a@example.test and owns order 1001.
- User B is user-b@example.test and owns order 1002.
- GET /orders/{order_id} checks authentication but deliberately skips the ownership check.
- A secure response to User A requesting order 1002 should be HTTP 403 or HTTP 404.
- The vulnerable route returns HTTP 200 with order 1002 and also exposes internal_notes and payment_reference.

EVIDENCE RULE

HTTP 200 alone does not prove BOLA. The returned JSON must identify User B's object and exactly match User B's owner baseline.

END-TO-END WORKFLOW

How SentinelAPI works

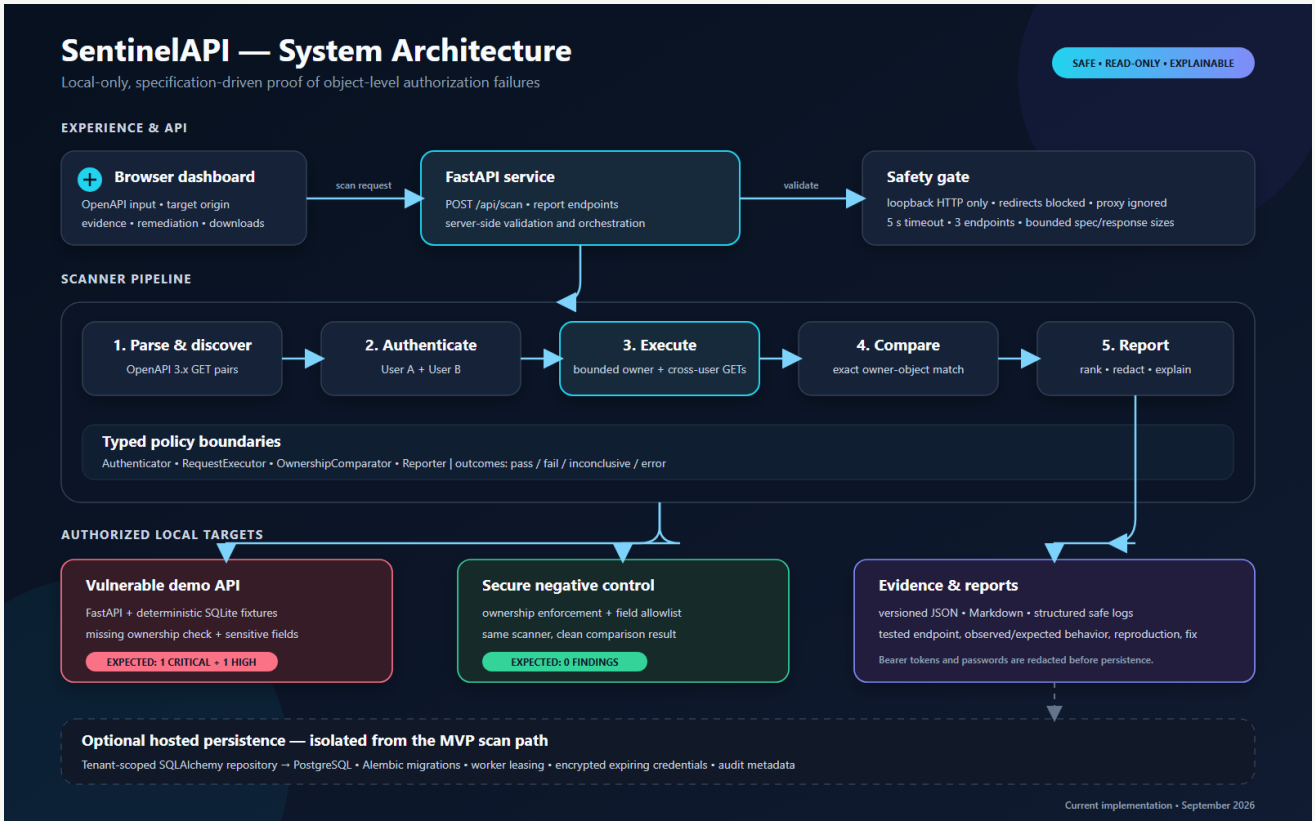
Step	Component	What happens
1	Dashboard	The engineer supplies a loopback API origin and an OpenAPI 3.x YAML or JSON document.
2	FastAPI route	The server validates input size and target origin, then invokes the scanner.
3	OpenAPI parser	The parser discovers up to three supported authenticated collection/detail GET pairs.
4	Authenticator	The scanner signs in as User A and User B through the controlled POST /auth/login flow.
5	Collection discovery	Each user's collection supplies one scalar object ID. The MVP assumes the first usable item belongs to that user.
6	Owner baselines	The scanner retrieves each selected object as its owner and verifies the returned ID.
7	Cross-user test	User A's bearer token requests User B's selected object ID.
8	Comparator	A confirmed BOLA finding requires HTTP 200 and an exact match with User B's owner response.
9	Exposure check	User A's own object response is inspected for known sensitive field names.
10	Reports and UI	The scanner ranks findings, redacts secrets, writes JSON and Markdown, and displays the evidence.

LIVE PROOF

User A owns 1001. User B owns 1002. User A receives 1002 from the vulnerable route.

TECHNICAL IMPLEMENTATION

System architecture



The browser is a presentation layer. The Python server enforces target validation, request limits, comparison rules, and redaction. The scanner pipeline keeps authentication, transport, comparison, checks, and reporting separate behind typed interfaces.

Component	Responsibility
app/	FastAPI sandbox, secure control, database lifecycle, scan route, downloads, and static dashboard.
scanner/openapi_parser.py	Safe OpenAPI parsing and bounded endpoint discovery.
scanner/authentication.py	Controlled two-user login and token acquisition.
scanner/request_execution.py	Timeout-bound local HTTP execution with redirect and proxy controls.
scanner/comparison.py	Ownership comparison policy and pass/fail/inconclusive decisions.
scanner/checks.py	BOLA-adjacent and sensitive-field checks.
scanner/core.py	End-to-end orchestration and local-target enforcement.
scanner/report_generator.py	Versioned JSON and Markdown evidence.
scanner/redaction.py	Secret-safe values for UI, logs, reports, and reproduction commands.

DECISION LOGIC

Detection policy and result states

A BOLA finding is confirmed only when all required conditions hold:

1. User A and User B have different selected object IDs.
2. Both owner detail requests return HTTP 200 JSON objects.
3. Each owner response matches the object ID selected from that user's collection.
4. User A's request for User B's object returns HTTP 200.
5. The cross-user JSON identifies User B's object and exactly equals User B's owner baseline.

Outcome	Meaning	Example
fail	The endpoint failed the security expectation and can produce a confirmed finding.	Cross-user HTTP 200 exactly matches User B's object.
pass	The API enforced ownership correctly.	Cross-user HTTP 403 or HTTP 404.
inconclusive	The scanner received an ambiguous result and cannot prove a vulnerability.	HTTP 429, identical IDs, or a mismatched HTTP 200.
error	A required step failed or the response was unusable.	Authentication failure, malformed JSON, timeout, or bad owner baseline.

Why exact matching?

Exact equality is conservative. It reduces unsupported findings but can miss a real issue when the API inserts dynamic fields such as timestamps. The MVP prefers a possible false negative over a false positive. Schema-aware comparison is future work.

How the exposure check works

The scanner walks response keys and normalizes common casing styles. It flags known sensitive names such as `internal_notes` and `payment_reference` but does not attempt full semantic data classification.

AUTHORIZED LOCAL TESTING

Safety and reliability controls

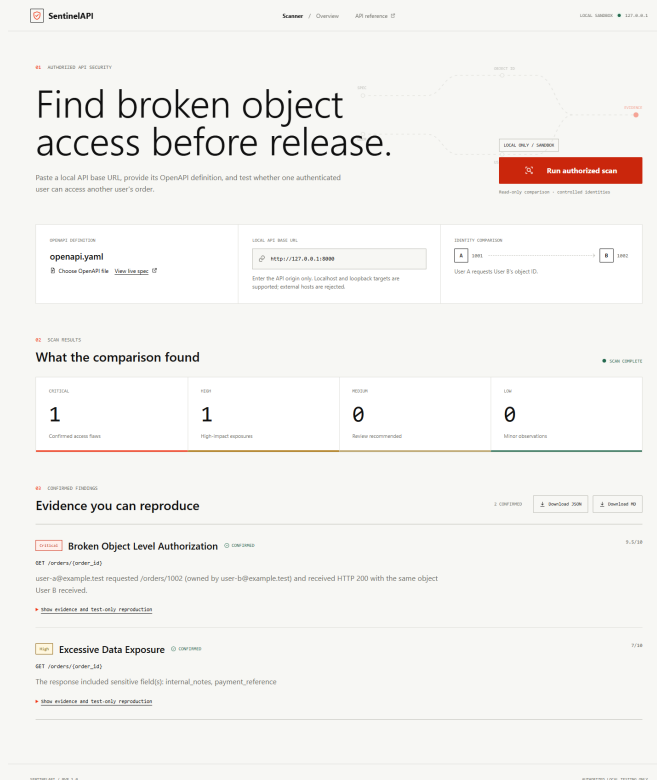
Control	Purpose
Loopback target policy	Accepts only HTTP origins on localhost, 127.0.0.1, and ::1.
Localhost normalization	Pins accepted localhost traffic to numeric loopback so DNS or host-file changes cannot redirect it.
Read-only scan path	Uses bounded GET requests. POST is used only for controlled demo authentication.
Final origin check	Rechecks scheme, host, and port immediately before transport dispatch.
Redirect and proxy controls	Blocks redirects and ignores environment proxy settings.
Request bounds	Five-second timeout, three-endpoint cap, 250 KB spec limit, and 1 MB response limit.
Safe parsing	Uses safe YAML loading and rejects dangerous or unsupported path forms.
Secret redaction	Removes passwords, tokens, and authorization values from logs, reports, errors, and reproduction.
Conservative evidence	Ambiguous, malformed, and failed comparisons cannot become confirmed findings.
Deterministic reset	Restores known local users and orders without deleting the SQLite file.

NON-NEGOTIABLE
BOUNDARY

Do not scan public, university, company, or production systems. Do not remove the loopback restriction to improve the demonstration.

USER EXPERIENCE

Dashboard and working prototype



The dashboard accepts a local origin and optional OpenAPI file, starts the scan, and displays the selected object IDs, cross-user result, severity counts, endpoint outcomes, evidence, reproduction, remediation, and report downloads. It distinguishes ready, running, complete, clean, inconclusive, and failed states.

**EXPECTED
VULNERABLE RESULT**

One Critical Broken Object Level Authorization finding and one High excessive-data-exposure finding.

Report contents

- Schema version and target summary.
- Severity-ranked findings and separate endpoint outcomes.
- Expected and observed behavior.
- Redacted test-only reproduction command.
- Direct remediation guidance.
- JSON for tooling and Markdown for human review.

FINDING ANATOMY

Explainable evidence

SentinelAPI

Scanner / Overview API reference

LOCAL SANDBOX 127.0.0.1

CONFIRMED FINDINGS

Evidence you can reproduce

2 CONFIRMED Download JSON Download MD

Critical

Broken Object Level Authorization

CONFIRMED

9.5/10

GET /orders/{order_id}

user-a@example.test requested /orders/1002 (owned by user-b@example.test) and received HTTP 200 with the same object User B received.

Show evidence and test-only reproduction

EXPECTED	OBSERVED
HTTP 403 or HTTP 404 for another user's object.	HTTP 200; object ID 1002 was returned.
REMEDIGATION	
Verify object ownership before returning order details.	

TEST-ONLY REPRODUCTION REQUEST

Copy request

curl -X GET "http://127.0.0.1:8080/orders/1002" -H "Authorization: Bearer <REDACTED>"

High

Excessive Data Exposure

CONFIRMED

7/10

GET /orders/{order_id}

The response included sensitive field(s): internal_notes, payment_reference

Show evidence and test-only reproduction

EXPECTED	OBSERVED
A normal user response should omit internal or secret fields.	Sensitive fields were returned in the API response.
REMEDIGATION	
Return an allowlisted response DTO for this endpoint and exclude internal notes, payment references, tokens, and secrets.	

TEST-ONLY REPRODUCTION REQUEST

Copy request

curl -X GET "http://127.0.0.1:8080/orders/1001" -H "Authorization: Bearer <REDACTED>"

The BOLA finding identifies the endpoint, requesting identity, target object, expected status, observed status, remediation, and a bearer-redacted curl command. The exposure finding identifies internal_notes and payment_reference without displaying secret values as evidence.

JUDGE-FACING EXPLANATION

A finding answers what was tested, what happened, why it is a security issue, what should have happened, how to reproduce it safely, and what to fix.

FALSE-POSITIVE CHECK

Secure negative control

02 SCAN RESULTS

What the comparison found

● SCAN COMPLETE

CRITICAL 0 Confirmed access flaws	HIGH 0 High-impact exposures	MEDIUM 0 Review recommended	LOW 0 Minor observations
---	--	---	--

The secure target scopes its database query by both object ID and current owner, returns HTTP 403 for a cross-user request, and allowlists response fields. The same scanner then produces a clean result with zero confirmed findings.

WHY IT MATTERS

The scanner distinguishes a protected API from the vulnerable sandbox instead of reporting a signature-based false positive.

Jury control specifications

File	Expected result
demo-specs/orders-operation-security.yaml	1 Critical BOLA and 1 High exposure finding.
demo-specs/orders-root-security.yaml	Clean result: HTTP 403 and zero findings.
demo-specs/orders-mixed-routes.yaml	Inconclusive result: HTTP 429 and zero false findings.

IMPLEMENTATION REFERENCE

Technology stack and repository map

Area	Technology	Purpose
Backend	Python, FastAPI, Pydantic	Scanner API, demo targets, validation, Swagger UI.
HTTP	HTTPX	Timeout-bound local requests with redirects and environment proxies disabled.
Specification	PyYAML and JSON	Safe OpenAPI 3.x ingestion and endpoint discovery.
Local storage	SQLite	Deterministic users and orders.
Dashboard	HTML, CSS, JavaScript	Dependency-free local interface with no trackers or remote fonts.
Reports	JSON and Markdown	Reproducible machine-readable and human-readable evidence.
Testing	unittest, TestClient, mock transport	Positive, negative, safety, redaction, reset, and report coverage.
Optional hosted layer	SQLAlchemy, Alembic, PostgreSQL, Argon2, Fernet	Tenant-scoped future persistence; separate from the active scan path.

Repository map

Path	Contents
app/	Local vulnerable and secure FastAPI applications, database seed, static dashboard.
scanner/	Parsing, authentication, execution, comparison, checks, models, redaction, reporting.
tests/	API, scanner, safety, reset, reports, and hosted persistence tests.
demo-specs/	Vulnerable, clean, and inconclusive jury-control specifications.
reports/	Latest generated JSON and Markdown outputs.
submission/	Decks, screenshots, architecture diagram, and submission copy.
migrations/ and app/hosted/	Optional hosted persistence foundation, not the active MVP scan route.

OPERATOR GUIDE

Setup, reset, test, and run

Initial setup

```
uv venv
.\.venv\Scripts\Activate.ps1
uv pip install -r requirements.txt
python -m app.seed
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Open <http://127.0.0.1:8000/> for the dashboard and <http://127.0.0.1:8000/docs> for Swagger UI.

Reset before a demo

```
.\.venv\Scripts\python.exe -m app.seed
```

CI-equivalent verification

```
.\.venv\Scripts\python.exe -m scripts.ci
```

Current verified result: 70 tests passed on 25 September 2026. Test counts can change as the project changes, so rerun the command before quoting a number.

Independent secure target

```
.\.venv\Scripts\python.exe -m uvicorn app.secure_main:app --host 127.0.0.1 --port 8011
```

For the most reliable seven-minute demo, use the three files under demo-specs against the main app on port 8000. Use the independent port 8011 target when the jury asks for a separate negative-control service.

IMPORTANT

Do not use `python -m scanner.demo` as proof of a live scan. That utility constructs an example report from predefined data.

DELIVERY SCRIPT

Seven-minute presentation plan

Time	Content	Presenter action
0:00-0:25	Opening	State the authorization problem and SentinelAPI's narrow claim.
0:25-0:55	Core functionality	Explain valid token versus object ownership, expected 403/404, and observed 200.
0:55-1:25	User journey	Describe target selection, endpoint discovery, two users, owner baselines, comparison, and report.
1:25-1:50	Key features	Group features into discovery, evidence-based testing, reporting, and negative control.
1:50-2:20	Technical implementation	Explain typed components and server-side safety boundaries.
2:20-5:40	Live demo	Run vulnerable scan, expand evidence, download report, then run secure control.
5:40-6:20	Verification	Show zero findings for secure behavior and mention the current automated test result.
6:20-7:00	Scope and close	State current limits, future work, and repeat the live proof claim.

Suggested opening

"APIs often verify that a user has logged in but fail to verify that the user owns the requested object. SentinelAPI tests this exact failure using live requests between two controlled users, then produces evidence developers can reproduce and fix."

Suggested closing

"Our claim is deliberately focused: SentinelAPI proves when one authenticated user receives another user's object from live API responses. The current MVP is narrow, safe, reproducible, and honest about what it does not yet support."

ON-STAGE FLOW

Live demo checklist and narration

Action	What the audience should see	What to say
Open the dashboard	Local origin 127.0.0.1:8000 and local-only label.	The scanner rejects external targets and remains inside the controlled sandbox.
Select orders-operation-security.yaml	The vulnerable /orders/{order_id} route.	The specification selects an authenticated collection/detail pair.
Run scan	User A object 1001, User B object 1002, HTTP 200 cross-user response.	The scanner established ownership before sending the cross-user request.
Open BOLA evidence	Expected 403/404, observed 200, object 1002, redacted reproduction.	The status and object both match the victim's owner baseline.
Open exposure evidence	internal_notes and payment_reference.	The owner response contains fields that should be excluded from a customer response.
Download report	JSON or Markdown file.	The same evidence is exported for engineering follow-up.
Select orders-root-security.yaml	HTTP 403 and zero findings.	The scanner runs unchanged. The API now enforces ownership correctly.

DEMO RULE

Never replace a failed live run with a screenshot while claiming that the screenshot is live. Label screenshots as evidence from a previous verified run.

CORE AND TECHNICAL QUESTIONS

Likely jury questions and answers

Question	Concise answer
What is the difference between authentication and authorization?	Authentication establishes who the caller is. Authorization decides which objects that caller may access.
How do you prove order 1002 belongs to User B?	User B's authenticated collection selects 1002, and the scanner retrieves 1002 as User B to establish the owner baseline.
Why two users?	Separate controlled identities provide separate ownership baselines and make cross-user access reproducible.
Does HTTP 200 automatically mean BOLA?	No. The cross-user response must identify User B's object and exactly equal User B's owner response.
How do you reduce false positives?	The scanner requires different IDs, valid owner baselines, exact object comparison, and treats 403/404 as correct behavior. Ambiguous results remain inconclusive.
Could exact comparison cause false negatives?	Yes, dynamic fields may prevent equality. The MVP deliberately prefers a possible false negative over an unsupported finding.
Are findings hardcoded?	The browser scan uses live authentication and HTTP responses. The seed data is deterministic, but the finding comes from the live comparison.
Why only localhost?	It prevents accidental or unauthorized scanning and keeps the intentionally vulnerable fixture off public networks.
Can it scan production?	No. The current product intentionally rejects non-loopback targets.
What OpenAPI shapes are supported?	A narrow OpenAPI 3.x collection/detail GET shape with an authenticated detail route and one path identifier.

SAFETY, SCOPE, AND DELIVERY

More jury questions and answers

Question	Concise answer
How is this different from a normal DAST tool?	It establishes ownership with two identities and compares owner and cross-user responses, targeting business-logic authorization failures.
How does exposure detection work?	It normalizes response key names and flags known sensitive fields. It is a bounded heuristic, not full semantic classification.
How are credentials protected?	Passwords, authorization headers, bearer values, and credential-shaped fields are redacted before output or persistence.
Why only GET requests?	The scanner proves read-access failures without changing the target database. Write testing requires additional safeguards.
Do you test rate limiting?	The core scanner does not perform stress testing. High-volume traffic is outside this safe MVP.
Is it production-ready?	No. It is a working local MVP with narrow OpenAPI and authentication support.
How can it scale later?	Typed boundaries let the team add authentication profiles, pagination, schema-aware comparison, and approved CI workflows without rewriting the UI.
What if both users receive the same ID?	The result is inconclusive because the scanner cannot establish separate ownership.
What if the target returns 429 or malformed JSON?	The scanner reports inconclusive or error rather than inventing a vulnerability.
Does AI make the security decision?	No. The active detection path is deterministic. AI assisted development and documentation, and the team must disclose and understand that assistance.
Why trust the implementation?	The current run has 70 passing tests covering the vulnerable flow, negative controls, safety limits, redaction, reset behavior, and reports.

FAILURE RECOVERY

Where the team can get stuck

Symptom	Likely cause	Recovery
Dashboard does not open	Server stopped, wrong environment, or port 8000 in use.	Start with .venv Python and uvicorn without --reload. Check the terminal for the actual port error.
Unexpected database data	A previous run or manual change altered the fixtures.	Run python -m app.seed before the session.
Zero endpoints discovered	Wrong OpenAPI file or unsupported security/path shape.	Select demo-specs/orders-operation-security.yaml and keep the origin at 127.0.0.1:8000.
Vulnerable scan shows zero findings	Secure spec selected, wrong route, stale server, or seed mismatch.	Verify the selected filename, reset the database, refresh, and run once more.
Secure scan shows findings	The vulnerable specification is still selected.	Select orders-root-security.yaml and confirm the endpoint is /secure-orders/{order_id}.
Report download unavailable	The scan has not reached a completed state.	Wait for completion, then download JSON or Markdown.
Old result remains visible	Browser state is stale after server restart or input change.	Refresh the page, reselect the spec, and run again.
Live scan fails on stage	Timeout, process restart, or malformed selection.	State that the run failed, try one reset and rerun, then switch to clearly labelled prior screenshots if necessary.
Jury asks for a public scan	Request conflicts with the safety boundary.	Refuse politely and explain that external targets are deliberately rejected.
Discussion drifts to hosted persistence	Optional foundation is being confused with the active product.	Clarify that the hosted database layer is separate and not wired into the current scan route.

Safe recovery line

"The current run did not complete, so we will not present it as a confirmed result. SentinelAPI treats failed execution as an error rather than manufacturing a finding."

HONEST PRODUCT BOUNDARY

Current limitations and future scope

Current limitations

- No public, production, university, or company API scanning.
- No HTTPS or non-loopback targets in the active MVP.
- No OpenAPI 2.x support, complete reference resolution, or general OpenAPI validation.
- Fixed demo authentication through POST /auth/login with two seeded users.
- No token refresh, pagination, nested resources, multiple identifiers, or query/header/cookie IDs.
- No write-operation testing, exploit chains, SQL injection, RCE, or load testing.
- The sensitive-field check is name-based and can miss semantically sensitive data.
- Exact response equality can miss vulnerabilities when legitimate responses include dynamic fields.
- No production deployment or active multi-tenant workflow.

Roadmap

Phase	Planned work
1. Stronger local scanner	Configurable local auth profiles, safe pagination, limited reference resolution, nested routes, response allowlists, and redacted diffs.
2. Hosted workflow	Connect the tested persistence foundation to an API and worker, then add RBAC, scan history, audit views, budgets, retention, and operations.
3. Authorized coverage	Additional deterministic authorization checks, schema-aware exposure detection, safe rate-limit configuration tests, and isolated CI integration.

OWNERSHIP

Team contributions, provenance, and submission cautions

Contributor	Commit-derived responsibilities	Demo responsibility
Divyanshu Saraswat	Problem statement record, scanner checks, reporting, safety architecture, secure control, documentation, and submission preparation.	Explain comparison policy, reports, secure control, safety boundaries, and architecture.
Yashneel Singh	Vulnerable sandbox, deterministic database, scanner orchestration, OpenAPI discovery, dashboard, reset, redaction, and safety testing.	Run reset and live demos; explain the API, database fixture, UI, and test evidence.

Facts requiring confirmation

- Both repository contributors must be confirmed as registered participants.
- A participant reviewer must be recorded for AI-assisted work.
- Organizer approval for any starter or pre-event material is not recorded in the workspace.
- The submission mechanism, cutoff, upload status, and presentation attendance require organizer confirmation.
- Team contribution descriptions must match the participants' actual work and understanding.

AI and open-source disclosure

OpenAI Codex assisted implementation and verification, including database hardening, hosted persistence foundations, automated tests, documentation, local browser verification, screenshots, and presentation artifacts. The active scanner does not use AI to decide whether a finding is confirmed. Registered participants remain responsible for review, understanding, attribution, and truthful submission.

**SUBMISSION
INTEGRITY**

Never claim production readiness, complete OWASP coverage, organizer approval, or participant ownership that has not been verified.

BEFORE JUDGING

Final quick reference

Terminal preparation

```
.\.venv\Scripts\python.exe -m scripts.ci  
.\.venv\Scripts\python.exe -m app.seed  
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Final checklist

- The dashboard opens at `http://127.0.0.1:8000/`.
- The vulnerable specification produces one Critical and one High finding.
- The secure specification produces HTTP 403 and zero findings.
- Both findings expand and the reproduction command is redacted.
- JSON or Markdown downloads successfully after a completed scan.
- No password, bearer token, or authorization secret appears in output.
- One presenter speaks while the other controls the demo.
- Screenshots remain a labelled fallback, not a substitute for a claimed live run.
- The team can explain every component, limitation, and contribution in its own words.

FINAL MESSAGE

SentinelAPI proves a narrow, high-impact API authorization failure with live evidence and a clean negative control. Its strength is reliable proof, bounded execution, and honest scope.

Primary repository sources: AGENTS.md, README.md, SOLUTION_OVERVIEW.md, ARCHITECTURE.md, SUBMISSION_CHECKLIST.md, TEAM_CONTRIBUTIONS.md, PROVENANCE.md, scanner source files, tests, reports, and submission screenshots.